

CM3015 ML&NN – Midterms 2024

Student Name: Devon Maya Xavier

UOL Student ID: 200618744

SIM Student ID: 10204322

Email: dmxavior001@mymail.sim.edu.sg

A Comparative Analysis of k-Nearest Neighbours and Decision Tree Algorithms for Mushroom Classification

TABLE OF CONTENTS

ABSTRACT	2
INTRODUCTION	2
BACKGROUND.....	2
<i>k</i> -Nearest Neighbours (<i>k</i> NN)	2
Decision Tree	3
METHODOLOGY.....	3
Dataset Preparation.....	3
Implementation of <i>k</i> NN	3
Implementation of Decision Tree.....	4
Evaluation Metrics	5
RESULTS.....	5
<i>k</i> NN Results.....	5
Decision Tree Results	6
Summary of Results.....	7
EVALUATION.....	7
CONCLUSIONS.....	8
REFERENCES.....	8

ABSTRACT

This project explores the application of two machine learning algorithms—**k-Nearest Neighbours (kNN)** and **Decision Tree**—to classify mushrooms as either edible or poisonous using the Mushroom Classification dataset. The dataset contains **8,124 samples** with **22 categorical features**, such as gill colour, cap shape, and odour. The kNN algorithm was implemented from scratch, while the Decision Tree was built using scikit-learn. Both models were evaluated using accuracy as the primary metric. The Decision Tree achieved a perfect accuracy of 100%, outperforming kNN, which achieved 95%. Feature importance analysis revealed that gill colour was the most significant factor in the Decision Tree's classification decisions. These findings highlight the strengths and limitations of each algorithm and provide insights into their suitability for categorical datasets.

INTRODUCTION

Machine learning algorithms play a critical role in solving classification problems, ranging from medical diagnostics to recommendation systems. In this project, we aim to classify mushrooms as either edible or poisonous using two popular machine learning algorithms: k-Nearest Neighbours (kNN) and Decision Tree. The goal is to compare their performance on the Mushroom Classification dataset, a public dataset from Kaggle.

The Mushroom Classification dataset is particularly well-suited for this study because it features clearly defined classes and categorical attributes that are relevant to real-world scenarios. Misclassifying a poisonous mushroom as edible could have serious consequences, underscoring the importance of accurate predictions. By analysing the performance of kNN and Decision Tree on this dataset, we can gain a deeper understanding of their respective strengths, weaknesses, and applicability to similar classification problems.

BACKGROUND

K-NEAREST NEIGHBOURS (KNN)

k-Nearest Neighbours (kNN) is a simple machine learning algorithm that classifies data points by looking at the **closest examples** in the training dataset. When given a new data point, kNN calculates how similar (or how “far away”) it is from all the data points in the training set, using a method like measuring straight-line distance (called Euclidean distance), which is applied in the methodology for this project.

After finding the **k nearest neighbours**—the closest examples—the algorithm looks at their classes (e.g., edible or poisonous) and predicts the class that appears most often among them. While kNN is easy to understand and use, it can become slow for very large datasets because it has to compare the new point to every single point in the training set. It also depends heavily on choosing the right number of neighbours (k) to get accurate predictions.

DECISION TREE

A Decision Tree is a machine learning algorithm that works like a flowchart. It classifies data by asking a series of **yes-or-no questions** about the features. For example, it might ask, "Is the gill colour white?" Depending on the answer, it moves to the next question, like "Does the mushroom have a foul odour?"

At each step, the algorithm picks the question (or feature) that best splits the data into groups where similar items (like edible or poisonous mushrooms) are together. It keeps splitting the data until everything is classified or until it reaches a stopping point, like a maximum number of questions.

Once the tree is built, it becomes very fast to use for predictions. You just follow the flow of questions down the tree to get the answer. Decision Trees are also easy to understand because you can see the rules it learned laid out in the tree structure.

METHODOLOGY

DATASET PREPARATION

The Mushroom Classification dataset was loaded and prepared for analysis. Since all features were categorical, they were converted into numerical values using label encoding. To ensure a representative distribution of the classes, the dataset was divided into two subsets: 80% for training the models and 20% for testing their performance.

```
print("Missing values:\n", df.isnull().sum()) # Check for missing values
df = df.dropna() # Drop rows with missing values (if any)

# Encode Categorical Features
label_encoders = {}
for column in df.columns:
    if df[column].dtype == 'object': # Check if the column is categorical
        le = LabelEncoder()
        df[column] = le.fit_transform(df[column])
        label_encoders[column] = le
```

```
] X = df.drop(columns=['class']) # Features
y = df['class'] # Target variable

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Training set shape:", X_train.shape)
print("Test set shape:", X_test.shape)
```

Training set shape: (6499, 22)

Test set shape: (1625, 22)

IMPLEMENTATION OF KNN

The kNN algorithm was implemented entirely from scratch in Python. For each test sample, the algorithm calculated the Euclidean distance to all training samples and identified the k -nearest neighbours ($k = 85$). The class most frequently represented among these neighbours was then assigned as the predicted class for the test sample.

```

# Function to calculate Euclidean distance
def euclidean_distance(x1, x2):
    return np.sqrt(np.sum((x1 - x2) ** 2))

# KNN Algorithm
class KNN:
    def __init__(self, k=81):
        # Ensure k is odd
        if k % 2 == 0:
            raise ValueError("k must be an odd number.")
        self.k = k

    def fit(self, X, y):
        # Input validation
        if len(X) != len(y):
            raise ValueError("The number of samples in X and y must be the same.")
        self.X_train = np.array(X)
        self.y_train = np.array(y)

    def predict(self, X):
        X = np.array(X)
        predictions = [self._predict(x) for x in X]
        return np.array(predictions)

    def _predict(self, x):
        # Compute distances
        distances = [euclidean_distance(x, x_train) for x_train in self.X_train]
        # Get indices of the k-nearest neighbors
        k_indices = np.argsort(distances)[:self.k]
        # Get the classes of the k-nearest neighbors
        k_neighbor_labels = [self.y_train[i] for i in k_indices]
        # Determine the majority class
        most_common = Counter(k_neighbor_labels).most_common(1)
        return most_common[0][0]

# Set a single value for k
k = 85 # Adjust this value as needed

```

IMPLEMENTATION OF DECISION TREE

A Decision Tree classifier was constructed using scikit-learn with default parameters, utilizing Gini impurity as the splitting criterion. The model was trained on the training dataset, and feature importance scores were analysed to identify the features that contributed most significantly to the classification process.

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

# Instantiate the Decision Tree Classifier
dt = DecisionTreeClassifier(random_state=42)

# Train the model
dt.fit(X_train, y_train)

# Predict on the test set
y_pred = dt.predict(X_test)

# Calculate and display accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Decision Tree Accuracy: {accuracy:.2f}")
print(f"Tree Depth: {dt.get_depth()}")
print(f"Number of Leaves: {dt.get_n_leaves()}")

# Visualize the Decision Tree
plt.figure(figsize=(18, 12))
plot_tree(
    dt,
    feature_names=X.columns,
    class_names=['Edible', 'Poisonous'],
    filled=True,
    fontsize=8
)
plt.title("Decision Tree Visualization", fontsize=16)
plt.show()

import pandas as pd
feature_importances = pd.Series(dt.feature_importances_, index=X.columns)
feature_importances = feature_importances.sort_values(ascending=False)

plt.figure(figsize=(10, 6))
feature_importances.plot(kind='bar')
plt.title("Feature Importance in Decision Tree")
plt.xlabel("Features")
plt.ylabel("Importance Score")
plt.show()

print(feature_importances)

```

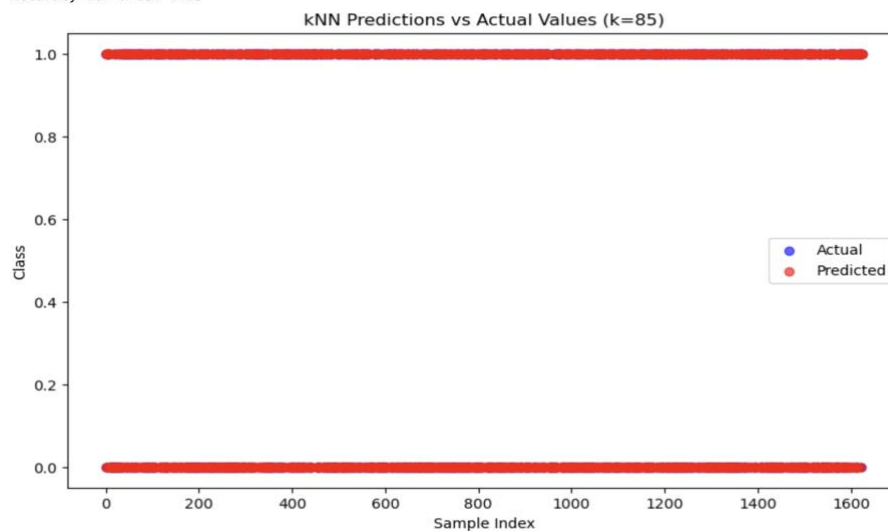
EVALUATION METRICS

Both models were primarily evaluated using **accuracy** as the key performance metric to determine their overall effectiveness in classifying mushrooms as either edible or poisonous. To gain a more comprehensive understanding of their performance, additional metrics such as **precision**, **recall**, and **F1-score** were calculated. These metrics provided deeper insights into the models' strengths, such as their ability to correctly identify poisonous mushrooms (recall) and their reliability in labelling mushrooms as edible or poisonous without errors (precision).

RESULTS

KNN RESULTS

Accuracy for k=85: 0.95



Precision: 0.99

Recall: 0.91

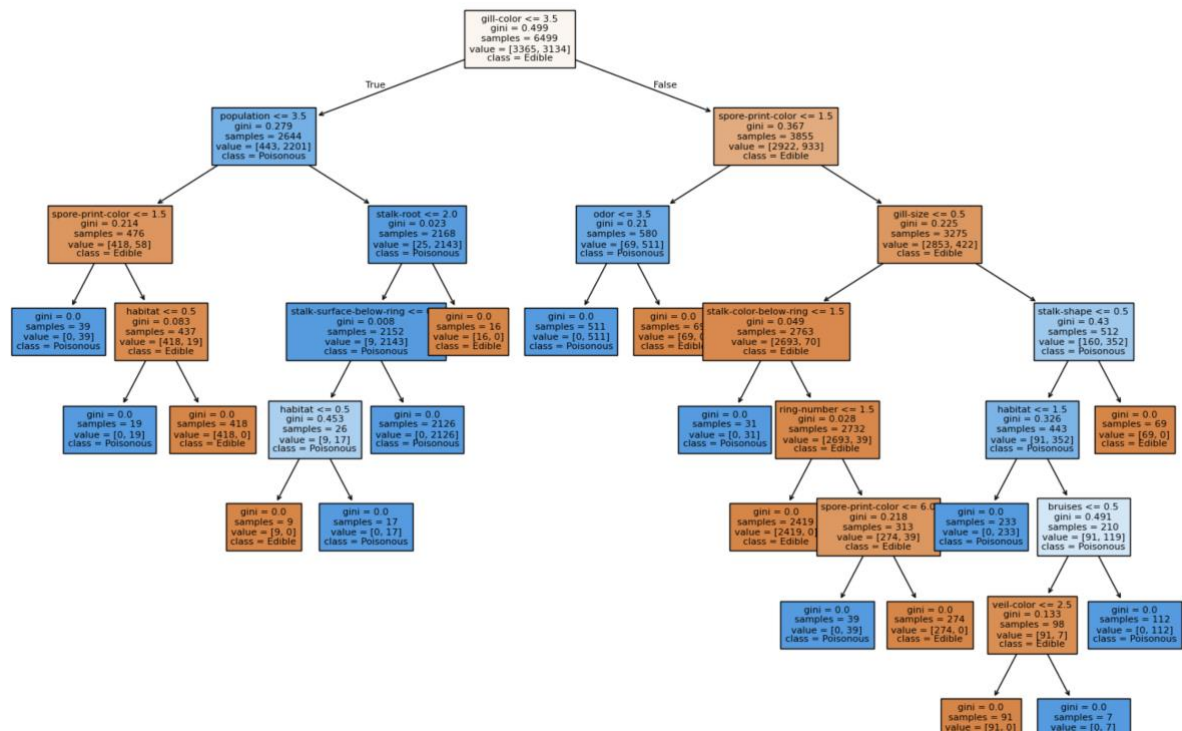
F1-Score: 0.95

	precision	recall	f1-score	support
0	0.92	0.99	0.95	843
1	0.99	0.91	0.95	782
accuracy			0.95	1625
macro avg	0.95	0.95	0.95	1625
weighted avg	0.95	0.95	0.95	1625

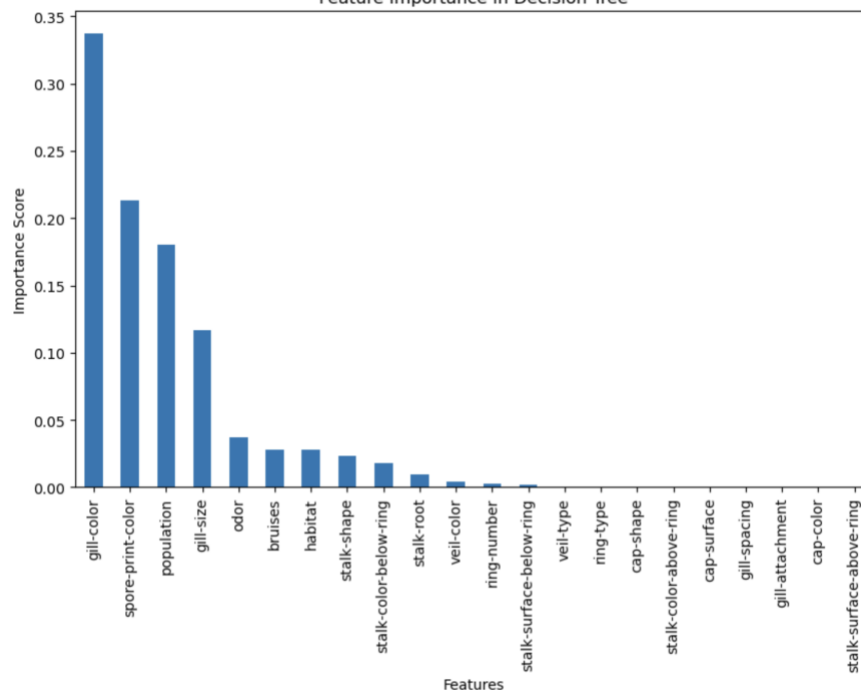
DECISION TREE RESULTS

Decision Tree Accuracy: 1.00
Tree Depth: 7
Number of Leaves: 18

Decision Tree Visualization



Feature Importance in Decision Tree



```

=== Decision Tree Evaluation Results ===
Accuracy: 1.00
Precision: 1.00
Recall: 1.00
F1-Score: 1.00

```

```

=== Classification Report ===
              precision    recall  f1-score   support

     0           1.00        1.00        1.00        843
     1           1.00        1.00        1.00        782

 accuracy          1.00          1.00          1.00        1625
 macro avg         1.00          1.00          1.00        1625
 weighted avg      1.00          1.00          1.00        1625

```

SUMMARY OF RESULTS

Metric	kNN	Decision Tree
Accuracy	95%	100%
Precision	99%	100%
Recall	91%	100%
F1-Score	95%	100%

The Decision Tree model achieved perfect accuracy, highlighting its capability to effectively learn and apply clear classification rules derived from the dataset. Analysing feature importance revealed that **gill colour** was the most influential feature, as it resulted in the greatest reduction in Gini impurity during data splits, offering the most distinct separation between edible and poisonous mushrooms.

In contrast, the kNN model also performed well, achieving a strong accuracy of 95%. However, it was slightly less accurate than the Decision Tree, primarily due to its reliance on distance-based calculations, which can be less effective for larger datasets. Additionally, kNN's sensitivity to the choice of k and the computational cost of comparing each test sample to all training samples contributed to its comparatively lower performance.

EVALUATION

The Decision Tree model's perfect accuracy underscores its suitability for the Mushroom Classification dataset. Its ability to ask sequential yes/no questions enabled the creation of precise classification rules, resulting in highly accurate and efficient predictions. However, this flawless performance may indicate **potential overfitting**, necessitating further validation through cross-validation methods to ensure the model's generalizability.

The kNN algorithm also delivered strong results, achieving an accuracy of 95%. But, its reliance on distance-based computations made it less effective for this dataset, which contains a large number of training samples. Moreover, kNN's sensitivity to the choice of k suggests that **additional optimization** could enhance its performance.

It is worth noting that the Mushroom Classification dataset may not have been the ideal choice for comparing these two algorithms. The **dataset's categorical nature and the clear separability of its classes heavily favour rule-based models like Decision Trees**, potentially placing kNN at a disadvantage. A dataset with more continuous features or overlapping classes might offer a more balanced comparison of these two algorithms.

CONCLUSIONS

This project compared the kNN and Decision Tree algorithms on the Mushroom Classification dataset, and I believe it has successfully achieved its aim. The Decision Tree emerged as the better model, achieving a perfect accuracy of 100%, while kNN achieved 95%. The Decision Tree's ability to learn rules from the data and make fast predictions proved advantageous, particularly for a dataset with clear feature-based splits. kNN, while slightly less accurate, remains a viable choice for smaller datasets or scenarios requiring simplicity.

To make these findings more robust, additional cross-validation techniques could be employed. For example, **k-fold cross-validation** could be used to evaluate the models across multiple data splits, providing a more reliable estimate of their generalization performance.

Additionally, for the Decision Tree model, **hyperparameter tuning** could be performed using methods like **grid search** or **randomized search** to optimize parameters such as `max_depth`, `min_samples_split`, and `min_samples_leaf`. This would ensure the model is not overfitting the training data while maintaining high accuracy on the test set. Pruning techniques could also be applied to reduce the complexity of the tree and improve its interpretability and generalizability. These additional evaluations and optimizations would strengthen the conclusions and provide deeper insights into the algorithms' capabilities.

In conclusion, the findings of this report **highlight the importance of selecting the right algorithm based on the dataset's characteristics and the application's requirements.**

REFERENCES

UCI Machine Learning Repository. (1987). *Mushroom Classification* [Data set]. Kaggle.
<https://www.kaggle.com/datasets/uciml/mushroom-classification>